Teaching Statement

Stanisław Szufa
University of Geneva · szufa.pl

Teaching Philosophy: Learning by Measuring

The most useful thing I learned during my PhD, I had to learn on my own. My advisor's work was theoretical, and when I wanted to know how voting rules actually behave, nobody around me could tell me how to design a numerical experiment, which data to use, or how to tell a real finding from an artifact of the generator. Working this out became my thesis, and then a tutorial, a community guide, and two software libraries, so that others would not have to start from scratch. This experience shapes how I teach. I want students to *measure* the objects they study, not only to prove things about them, because a student who has computed a Kemeny ranking on a real election and watched the ILP solver stall on a specific kind of profile understands NP-hardness in a way that no reduction on a blackboard can give.

In practice, this means three things in my courses:

- *Every abstraction comes with an instance.* I introduce a definition together with a dataset or a generator on which students can try it in the same session. In my field this is easy: a city's participatory budget is a real election with real stakes, and students can load one from Pabulib, run three voting rules on it, and argue about which outcome is fairer before they have seen a single axiom.
- *Students should discover, not only verify.* The exercises I like best have an answer that I do not know in advance. "Which statistical culture best approximates the preferences of Warsaw's residents?" is a question a second-year student can attack with the tools from my research, and the answers are sometimes publishable.
- *Tools are part of the curriculum.* Writing clean, reproducible experimental code is a skill that most students are never taught. I teach it explicitly, using `prefsampling` and `mapel` as both examples and infrastructure, and I ask students to contribute to them instead of writing throwaway scripts.

Teaching Experience

As a Teaching and Research Assistant at Jagiellonian University (2018–2021), I taught the exercise sessions of three core bachelor courses in computer science: *Introduction to Programming*, *Logic and Set Theory*, and *Linear Algebra*. These are first- and second-year courses with large cohorts and a wide spread of preparation, and they taught me to explain the same idea in several ways and to notice early which students are falling behind. They also cover the material I would most like to teach at the bachelor level at EPFL: programming, discrete mathematics, and algorithms are the foundation on which everything in my research stands.

Beyond university courses, I have taught my research community. I gave a tutorial on numerical experiments in computational social choice at three of the major AI conferences, AAAI, AAMAS, and IJCAI, and I co-authored the *Guide to Numerical Experiments on Elections* (IJCAI-24), which is written as teaching material: it surveys how experiments have been done, explains what goes wrong, and gives concrete recommendations. The two open-source libraries I maintain, `prefsampling` and `mapel`, are used by other groups both for research and in student projects. I have also given more than twenty talks at conferences and seminars, and explaining voting rules to city officials and residents through *Science for Democracy* has been the most demanding teaching I have done: the audience has no patience for notation and every question is about a real decision.

Mentoring and Supervision

I have co-supervised more than ten bachelor's and master's students, mostly at AGH University of Kraków, within an ERC-funded project, and at LAMSADE in Paris. Several of these projects grew into publications at AAAI, AAMAS, ECAI, and NeurIPS with the student as a coauthor. My approach follows the way I was mentored and the way I would have liked to be:

- *Start small and concrete.* A student's first project is a well-defined experimental question with a dataset and a tool already in hand, so that the first result comes within weeks. Confidence comes from a finished thing, however small.
- *Then hand over an open problem.* Once a student has shipped something, I give them a question whose answer I do not know, and I expect them to disagree with me. The best paper award I received at AAMAS-23 was for a paper written by three PhD students without a senior coauthor, and I want my own students to have that experience.
- *Teach the craft, not only the result.* I review students' code and experimental design as carefully as their proofs, and I ask them to present their work to the group early and often. Writing clearly and presenting well are skills I teach deliberately, because they decide whether good work is noticed.

As President of the PhD Students' Association at Jagiellonian University and as a representative in the Polish national association of doctoral candidates, I spent two years working on the conditions under which doctoral students work: funding, supervision rules, and representation in university bodies. I know from that work how much a student's success depends on things outside the research itself, and I will pay attention to them in my group.

Teaching at EPFL

Existing courses. I am prepared to teach core courses at the bachelor level, in particular introductory programming, discrete mathematics, algorithms, and theory of computation, and at the master level, artificial intelligence and algorithmic game theory. I speak French at a conversational level and am improving it actively; I expect to teach bachelor courses in French within my first years at EPFL.

New courses. I would like to develop two courses that do not yet exist in most computer science curricula:

- *Algorithms for Collective Decisions* (master level). Voting, participatory budgeting, fair division, and matching, taught through the lens of my research: every rule comes with real data, every axiom with an experiment, and the project is a full study of a real participatory process, from data to recommendation. The course would draw students from computer science, data science, and the social sciences.
- *AI and Democracy* (master seminar). How language models change collective decision-making: AI-mediated deliberation, LLM proxies, representation in generated text, and the risks of each. The seminar would combine recent papers with hands-on simulation, and it would be a natural meeting point for students of the School of Computer and Communication Sciences and of the humanities and social sciences at EPFL.

Beyond the classroom. Through Science for Democracy I have learned how to explain algorithms to people who will be governed by them, and I would bring this to EPFL: student projects with Swiss municipalities that run participatory budgets, public explanations of how their votes are counted, and open tools that residents can use. Teaching people why a proportional rule funded one project and not another is, to me, the same job as teaching students why an algorithm is correct.